

Relazione SQL Injection HW

Esame di Sicurezza

Lorenzo Zanda (2006432)

Agosto 2024

Indice

1	Introduzione	2
2	Codice	2
3	Vulnerabilità	3
3.1	Costruzione delle query	3
3.2	Errori troppo dettagliati	3
4	Dimostrazione pratica	4
4.1	Funzionamento corretto	4
4.2	Attacchi SQLi	6
4.2.1	Tautology	6
4.2.2	EOL comment	6
4.2.3	Piggybacked queries	7
4.2.4	Proprietà CIA violate	8
5	Considerazioni	9

1 Introduzione

Questo progetto è stato realizzato con lo scopo di dimostrare le vulnerabilità di un'applicazione web ad attacchi di *SQL injection*.

L'**SQL injection** è una tecnica di **hacking** che permette, sfruttando degli errori di programmazione, di inserire **comandi SQL** all'interno di una **query** destinata al **database**. Le conseguenze di un attacco del genere possono essere **devastanti**, si tratta di conseguenze quali l'**accesso non autorizzato a dati sensibili**, la **manipolazione dei dati** o addirittura la **cancellazione di intere tabelle**.

L'**obiettivo** principale di questo progetto è mostrare proprio come un'applicazione web, se non programmata correttamente, possa essere **vulnerabile** a questo tipo di **attacchi**.

L'applicazione sviluppata è semplice e consta di due schermate principali:

- **Schermata di login:** Dove l'utente inserisce le proprie credenziali (**username** e **password**) per accedere al sistema.
- **Schermata di login avvenuto con successo:** Visualizzata dopo un login corretto, confermando che l'utente ha inserito credenziali valide.

Le **tecnologie** utilizzate per lo sviluppo sono le seguenti:

- **PHP 7.4:** Utilizzato per sviluppare il backend dell'applicazione e l'interazione col database.
- **Apache:** Server web per gestire le richieste HTTP.
- **MySQL 5.7:** DBMS utilizzato per la gestione dei dati dell'applicazione.
- **Docker:** Utilizzato per creare un ambiente di sviluppo isolato che garantisca la portabilità del progetto.

2 Codice

Il progetto è strutturato come segue:

- **database/:** Contiene lo script **database.sql** che crea il database e la tabella **users**.
- **src/:** Contiene al proprio interno tutti i file PHP del progetto, tra cui il file **login.php** che gestisce l'autenticazione degli utenti e presenta vulnerabilità a SQL injections.
- **Dockerfile:** Definisce l'immagine per il servizio web basata su PHP e Apache.

- `docker-compose.yml`: File di configurazione Docker per avviare il server web e il database.

Il **codice**, con le relative indicazioni per il suo funzionamento, è accessibile tramite il seguente link: [Repository GitHub](#)

3 Vulnerabilità

Vediamo ora i principali motivi per cui il programma è **vulnerabile** ad attacchi **SQL injection**.

3.1 Costruzione delle query

La **costruzione della query** in `login.php` include **input** dell'utente senza **adeguate misure di sicurezza**, l'attaccante può così manipolare l'input per **alterare** la logica della query e **sfruttare il sistema** a suo piacimento.

```
$user = $_POST['username'];
$pass = $_POST['password'];

// Query SQL vulnerabile a SQL Injection
$sql = "SELECT * FROM users WHERE username = '$user' AND password = '$pass'";
```

Strutturando il codice in questo modo, i valori di `$user` e `$pass` vengono inseriti **direttamente** nella **query SQL**, rendendo l'applicazione **vulnerabile**.

3.2 Errori troppo dettagliati

Il codice attuale rivela **informazioni sensibili** tramite **messaggi di errore** troppo dettagliati, un aspetto che permette a utenti malintenzionati di conoscere nomi di **colonne** o **tabelle**.

```
$_SESSION['error'] = "Invalid username or password:
                      no match found in table users";
$_SESSION['error'] = "SQL Error: " . $conn->error;
```

4 Dimostrazione pratica

Vediamo ora più nel dettaglio il comportamento dell'applicazione effettuando dei veri e propri **attacchi inband**.

4.1 Funzionamento corretto

La tabella **users** viene inizializzata con tre utenti ed inserendo **correttamente** le credenziali, il login avrà perfettamente **successo**.

```
mysql> SELECT * FROM users;
+----+-----+-----+
| id | username | password |
+----+-----+-----+
| 1  | user    | password |
| 2  | test    | secret-password |
| 3  | admin   | admin    |
+----+-----+-----+
3 rows in set (0.00 sec)
```

Figura 1: La tabella users

Proviamo ad eseguire l'accesso come **admin** utilizzando le corrette credenziali.

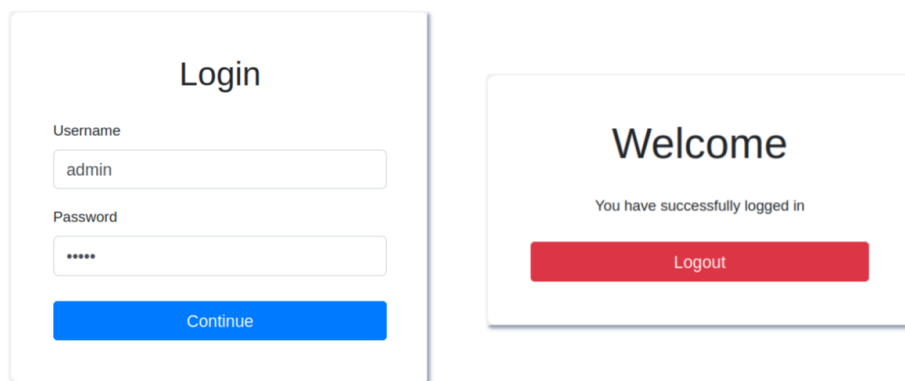
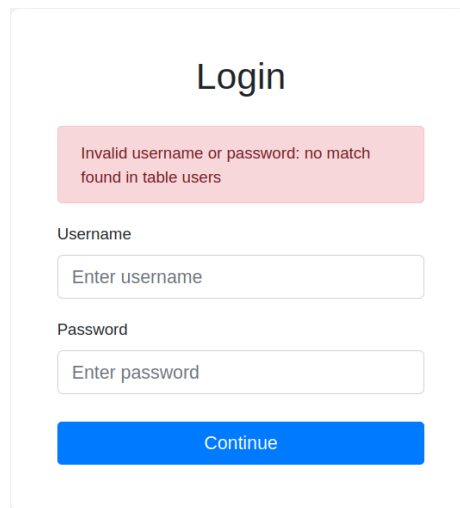


Figura 2: Login effettuato

Nel caso in cui le credenziali siano errate, il login **non avrà successo** e verrà mostrato un **messaggio d'errore**.

Inseriamo ad esempio una password errata per l'utente **admin**



The image shows a web form titled "Login". At the top, there is a red error message box that reads: "Invalid username or password: no match found in table users". Below this, there are two input fields: "Username" with the placeholder text "Enter username", and "Password" with the placeholder text "Enter password". At the bottom of the form is a blue button labeled "Continue".

Figura 3: Login fallito

Analizzando il **messaggio d'errore** scopriamo inoltre dell'esistenza della tabella **users**, informazione che ci tornerà utile per **attaccare il sistema** successivamente.

4.2 Attacchi SQLi

4.2.1 Tautology

Questo tipo di attacco si verifica quando un utente inserisce **istruzioni condizionali** in modo che vengano valutate **sempre come vere**.

Ad esempio si supponga che l'utente inserisca `'admin' OR '1'='1'` nel campo username.

La query risultante sarà:

```
SELECT * FROM users WHERE username = 'admin' OR '1'='1' AND password = '';
```

Poiché la condizione `'1'='1'` è sempre vera, questo input permette all'utente di effettuare il login come `admin` ma bypassando completamente la **verifica della password**.

Se testato sull'applicazione web, il login **avverrà con successo**.

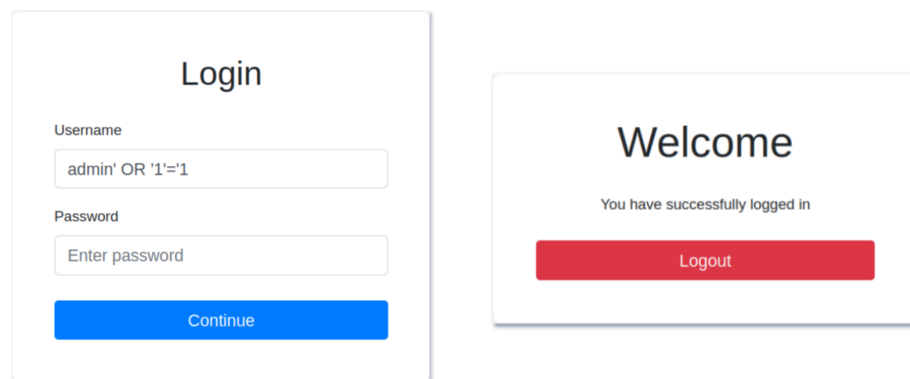


Figura 4: Esempio di tautology

4.2.2 EOL comment

In questo tipo di attacco si fa utilizzo di un **commento a fine riga** (`--`) in modo che tutto ciò che viene **dopo** il commento **venga ignorato**.

Si supponga che l'utente inserisca nel campo username un input del genere `' OR '1'='1' --`

La query risultante sarà:

```
SELECT * FROM users WHERE username = '' OR '1'='1' -- AND password = '';
```

L'input **disabilita** la **verifica della password** e utilizza l'istruzione condizionale sempre vera già usata nell'esempio della **Tautology** (4.2.1). In questo caso però le **conseguenze** sono ancora più **disastrose**: il database restituisce tutte le righe della tabella `users` indipendentemente dai valori dei campi username e password.

Così come nell'esempio precedente, l'applicazione web permette di effettuare l'attacco.

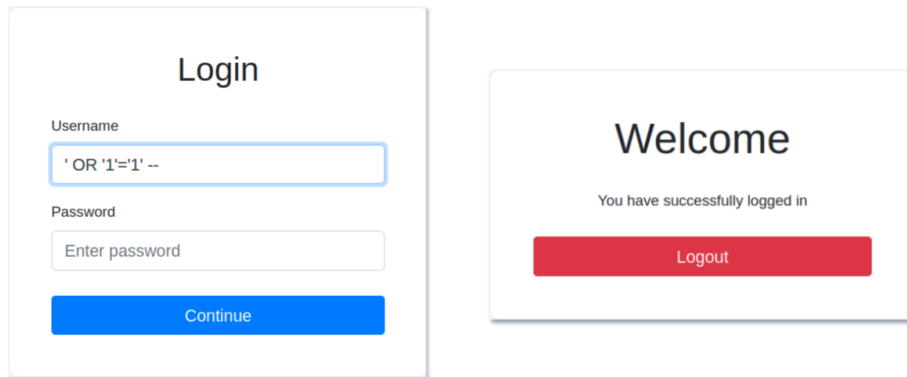


Figura 5: Esempio di EOL

4.2.3 Piggybacked queries

A differenza degli altri due, un attacco SQL injection di questa tipologia permette all'attaccante di eseguire **query multiple** in una **sola istruzione SQL**.

Tutto ciò non sarebbe possibile se l'applicazione web non fosse progettata per performare query multiple sul database ma, sfortunatamente, o fortunatamente per il solo utente malintenzionato, il nostro programma supporta **query multiple** tramite l'utilizzo della funzione PHP `multi_query()`.

Inseriamo `''; DROP TABLE users; --` nel campo username.

La query risultante sarà:

```
SELECT * FROM users WHERE username = ''; DROP TABLE users; -- ' AND password = '';
```

Questa query esegue la cancellazione della tabella `users`, dimostrando come un attacco di questo tipo possa essere **estremamente dannoso**.

Replicando l'attacco sull'applicazione e andando a controllare lo stato del database, possiamo constatare che la tabella **users** non è più esistente.

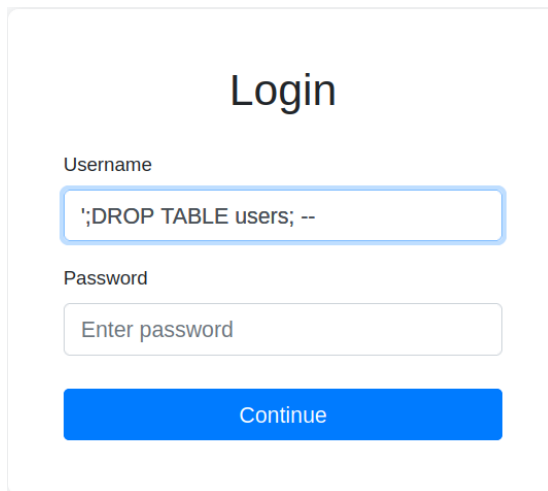


Figura 6: Esempio di piggybacked query

```
mysql> SELECT * FROM users;  
ERROR 1146 (42S02): Table 'sql_db.users' doesn't exist
```

Figura 7: La tabella users è stata eliminata in seguito all'attacco

4.2.4 Proprietà CIA violate

Gli attacchi SQLi presentati violano le seguenti **proprietà CIA**:

- **Confidenzialità (Confidentiality):** L'accesso non autorizzato a dati sensibili, come le credenziali degli utenti, viola la **confidenzialità**, esponendo informazioni riservate.
- **Integrità (Integrity):** Gli attacchi che manipolano o eliminano i dati in modo non autorizzato, compromettono l'**integrità** del database, alterando o distruggendo informazioni critiche.
- **Disponibilità (Availability):** La cancellazione della tabella **users** mediante un **attacco piggybacked** può rendere l'intera **applicazione inutilizzabile**.

5 Considerazioni

Questo progetto ha dimostrato in modo pratico le vulnerabilità di un'applicazione web agli attacchi SQL injection evidenziando la **gravità** delle **conseguenze** degli attacchi. E' **importante** dunque **implementare** sempre delle **misure preventive** come l'uso di **prepared statements** e mantenere un'**attenzione costante** alla sicurezza durante lo sviluppo.

Riferimenti bibliografici

- [1] William Stallings and Lawrie Brown. *Computer security: principles and practice*. Pearson, 2018.